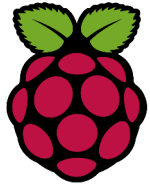


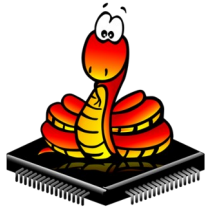


Atelier Raspi

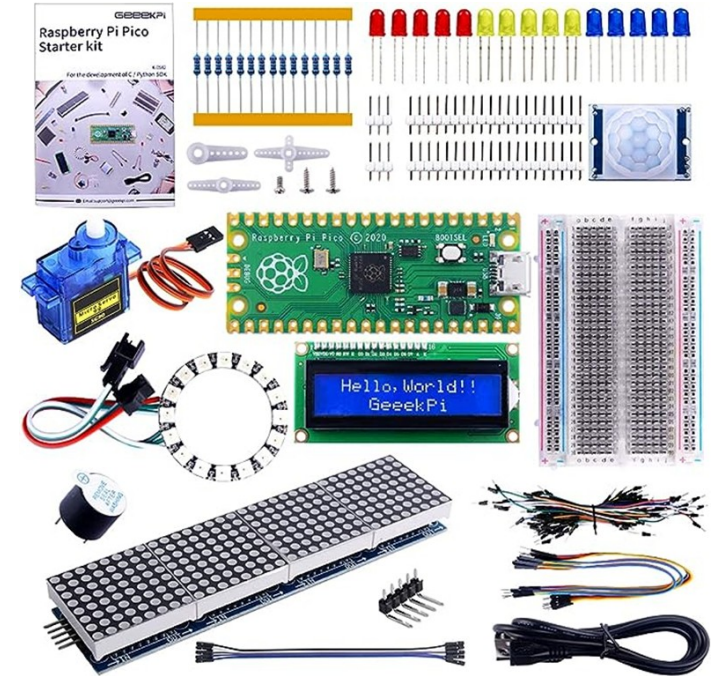
Atelier N°18 La mini serre instrumentée Boucle de Régulation



Logo du Raspberry Pico



Logo du MicroPython

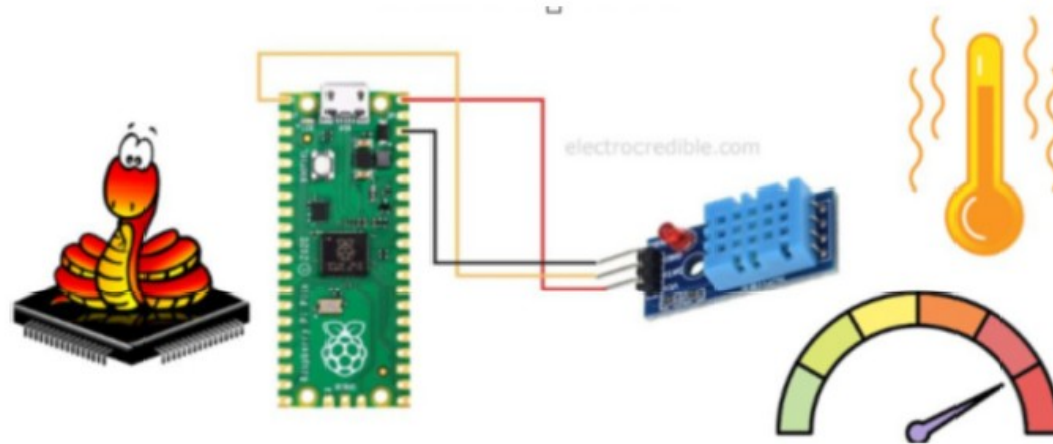


L'atelier a pour valeurs, le partage, l'aide, la formation, le faire et construire ensemble à partir de l'expérience des participants

Atelier N°18 La mini serre instrumentée

Boucle de Régulation

- Etape 1 Rappel des éléments du projet
- Etape 2 Rappel de la séance précédente
- Etape 3 Les boucles de régulation
- Etape 4 La régulation d'humidité
- Etape 5 Le programme (1/2)
- Etape 6 Le programme (2/2)
- GI1 Glossaire informatique1
- GI2 Glossaire informatique2
- GI3 Glossaire informatique3
- GI4 Glossaire informatique4
- GI5 Glossaire informatique5
- GI6 Glossaire informatique6
- GE1 Glossaire électronique



A18E1 Rappel des éléments du projet

Le cerveau de la serre

Carte MicroPython → le “cerveau” du système 🤖

Elle exécute le programme et prend les décisions.

Le module interface homme-machine

Il permet de communiquer avec la serre :

Écran LCD → affiche les informations

4 boutons → envoyer des commandes

4 LEDs → indiquer des états

Ruban de 8 LEDs → afficher des niveaux ou des animations 🌈

Les capteurs : DHT11 ou DHT22

Ces capteurs mesurent la température 🌡️ et l'humidité 💧

La carte MicroPython lit ces informations pour savoir comment réagir.

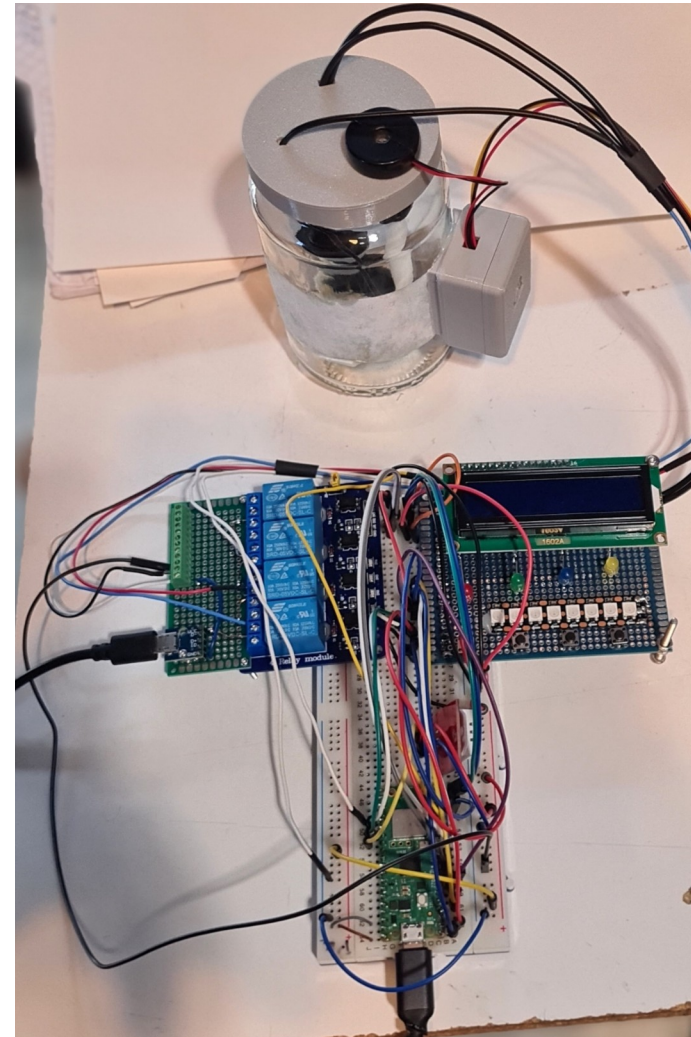
Les actionneurs : Carte 4 relais

Les relais permettent d'allumer ou d'éteindre des appareils électriques.

Relais du brumisateurs → ajoute de l'humidité 💧

Relais de la pompe → fait circuler l'eau 🌊

Relais du ventilateur → enlève de l'humidité



*Nous n'utiliserons pas le servo moteur qui ouvre une trappe
et le détecteur de présence vu pour le sapin de Noël.*

A18E2 Rappel de la séance précédente

Transition avec la séance précédente

Lors de la séance précédente, nous avons utilisé les boutons BP1 et BP2 pour tester les relais et commander la pompe manuellement.

BP1 → allumer la pompe

BP2 → arrêter la pompe

Le programme fonctionnait comme une petite machine à états :

état « éteint »

état « allumé »

👉 C'était l'utilisateur qui décidait quand changer d'état en appuyant sur les boutons.

Aujourd'hui, on change de fonctionnement

Cette fois, les boutons ne vont plus commander directement les relais.

C'est le programme qui prendra les décisions automatiquement grâce aux mesures du capteur d'humidité.

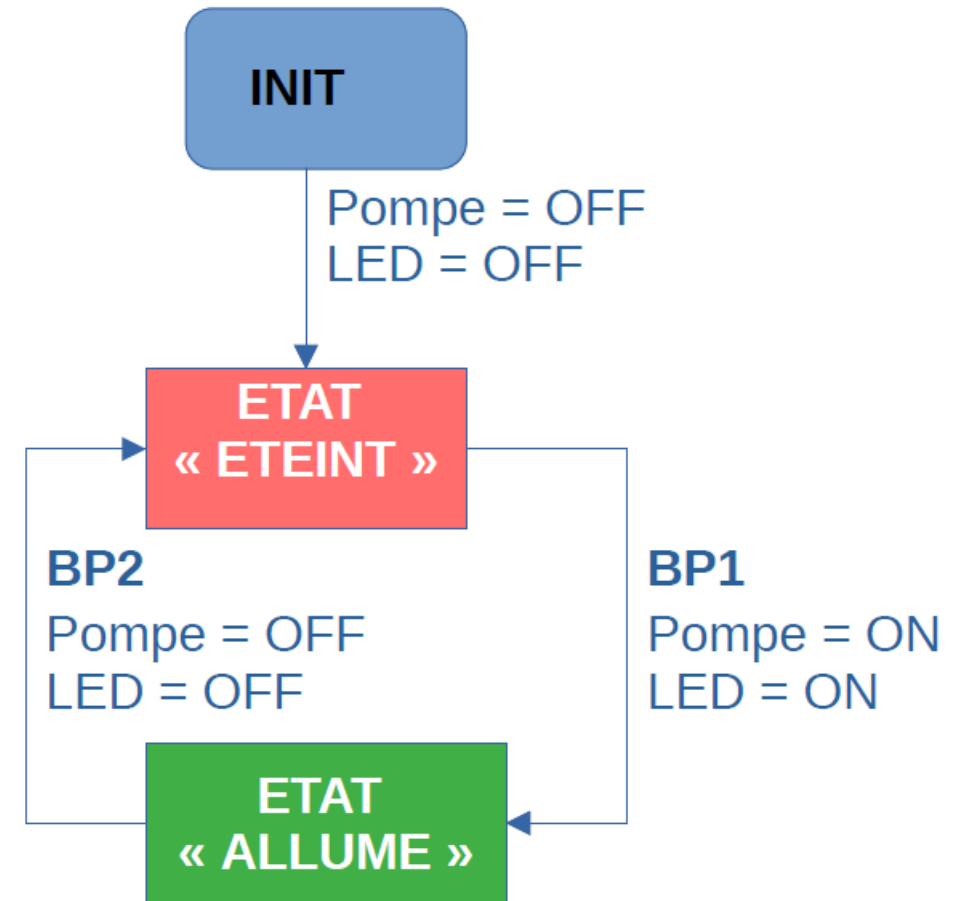
Au lieu de :

« J'appuie sur un bouton pour allumer »

le programme fera :

« L'air est trop sec, donc j'allume le brumisateurs »

Test du relais / pompe avec BP1 et BP2



A18E3 Les boucles de régulation

On passe à une boucle de régulation 💧

Le programme va maintenant :

- mesurer l'humidité,
- comparer avec des seuils,
- décider quoi faire,
- commander automatiquement les relais.

La serre devient alors autonome 🌱🤖

On peut créer plusieurs régulations :

Régulation de température 🌡️

Si la température est trop haute → on active le ventilateur

Si la température est trop basse → on pourrait activer un chauffage

Régulation d'humidité 💧

Si l'air est trop sec → on active le brumisateur

Si l'air est trop humide → on active le ventilateur

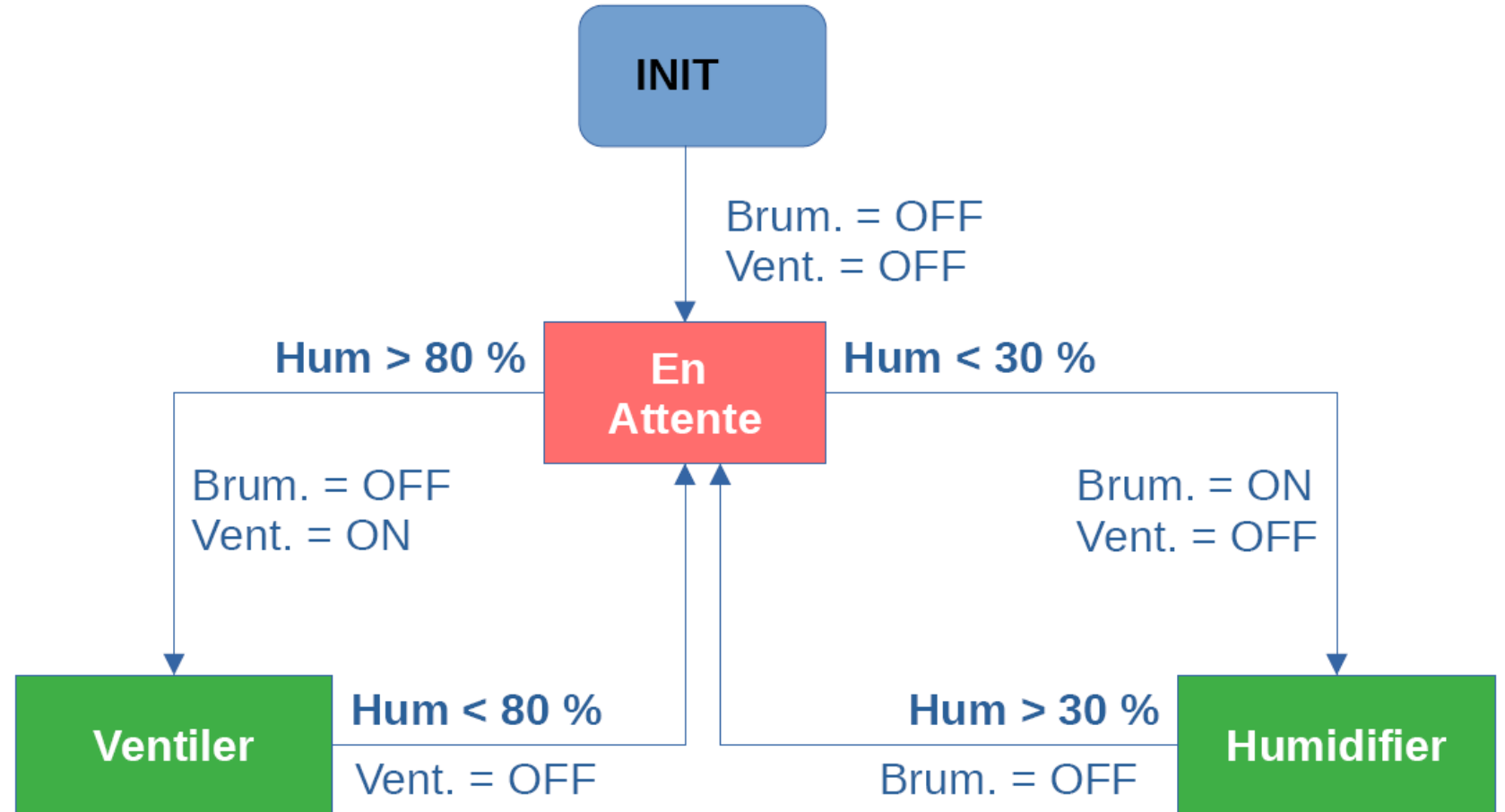
A18E4 La régulation d'humidité

Si l'air est trop sec → on active le brumisateurs

Si l'air est trop humide → on active le ventilateur

Notre programme va :

- mesurer l'humidité de l'air,
- décider quoi faire,
- piloter automatiquement les relais. 🤖



A18E5 Le programme (1/2)

```
1  from machine import Pin
2  import time
3  import dht
4
5  time.sleep(0.1) # Wait for USB to become ready
6
7  # CONSTANCE
8  # definit les seuils d'humidite bas et haut
9  SEUIL_HUMIDITE_BAS=30
10 SEUIL_HUMIDITE_HAUT=80
11
12 # FONCTION
13 def afficher_humidite( valeur ):
14     print(f"humidite : {valeur:.1f}%")
15     # A FAIRE utiliser le bandeau de led
16
17 # PROGRAMME PRINCIPAL
18
19 print("Boucle regulation humidite !")
20
21 # INIT
22 # initiliser le bandeau led
23 relais4_ventilateur = Pin(9, Pin.OUT)
24 relais2_brumisateur = Pin(7, Pin.OUT)
25 #DHT = dht.DHT11(Pin(22))
26 DHT = dht.DHT22(Pin(22))
27
28 # place les actionneur dans leur etat par default
29 relais4_ventilateur.value(1) # eteint le brumisateur
30 relais2_brumisateur.value(1) # eteint la pompe
31 mon_etat = "attente" # variable d'etat du programme : "humidifie" / "attente" / "ventille"
32
```

A18E6 Le programme (2/2)

```
33     # boucle principale
34     while True:
35         # on prend la mesure
36         DHT.measure()
37         # on affiche le niveau d'humidite
38         afficher_humidite( DHT.humidity() )
39         # en fonction de notre etat
40         if mon_etat == "attente":
41             # si l'humidite est en dessous du seuil bas
42             if DHT.humidity() < SEUIL_HUMIDITE_BAS:
43                 # on allume le brumisateurs
44                 relais2_brumisateurs.value(0)
45                 # on eteint le ventilateur
46                 relais4_ventilateur.value(1)
47                 # on indique notre nouvel etat
48                 mon_etat = "humidifie"
49             elif DHT.humidity() > SEUIL_HUMIDITE_HAUT:
50                 # on eteint le brumisateurs
51                 relais2_brumisateurs.value(1)
52                 # on allume le ventilateur
53                 relais4_ventilateur.value(0)
54                 # on indique notre nouvel etat
55                 mon_etat = "ventille"
56         elif mon_etat == "humidifie":
57             # on regarde si on est passe au dessus du seuil d'humidite pour arreter le brumisateurs
58             if DHT.humidity() > SEUIL_HUMIDITE_BAS:
59                 # on eteint le brumisateurs
60                 relais2_brumisateurs.value(1)
61                 # on indique notre nouvel etat
62                 mon_etat = "attente"
63         elif mon_etat == "ventille":
64             # on regarde si on est passe en dessous du seuil d'humidite pour arreter le ventilateur
65             if DHT.humidity() < SEUIL_HUMIDITE_HAUT:
66                 # on eteint le ventilateur
67                 relais4_ventilateur.value(1)
68                 # on indique notre nouvel etat
69                 mon_etat = "attente"
70
```


G11 Glossaire informatique1

Instruction du langage : ne nécessite pas d'importer de module pour les utiliser

print('Bonjour') : affiche Bonjour sur la console du PC

While test : Boucle tant que test est VRAI

import : pour importer des modules contenant des fonctions

import time : permet d'appeler les fonctions relatives au temps

time.sleep(x) : arrête l'exécution du programme pendant x secondes

import machine : module contenant les fonctions permettant d'agir sur les périphériques du pi pico

from : pour simplifier l'écriture on importe certaines fonctions directement dans une variable à l'aide de « from »

from machine import Pin : permet d'accéder à la fonction Pin du module machine uniquement.

ma_led_verte = Pin(n, Pin.OUT) : permet d'indiquer comment on souhaite utiliser un GPIO,
ici on a configuré le p I/O N° n en sortie tout ou rien

ma_led_verte.Toggle() : inverse l'état de l'I/O associé à la variable.

Ici l'IO n passera de l'état haut (+3.3v) à l'état bas (0v) ou inversement

: En début de ligne permet d'écrire un commentaire, ce qui est écrit n'est pas une instruction et donc n'est pas pris en compte par le programme.

Variable A=A+1 ->Nouvelle valeur = ancienne valeur+1 , peut s'écrire A+=1

print (" texte " , A), input ("texte" , B)

Opérateurs: +,-,*,/,%, **Données**: int(), float(), str(), bool(=True or False

G|2 Glossaire informatique2

Liste -> Création d'une liste, `liste_de_noms = ['Toto', 'Hubert', 'Jacques', 'Philippe']`

`liste_quelconque = ['Toto', 1, 'Philippe', 4,5,6]`

`print ('Nombre d'articles dans la liste : ', len(liste_quelconque))`

`for items in liste_quelconque:`

`print (items)` -> parcourt la liste et imprime chaque item de la liste

Pour récupérer un élément de la liste, **variable= liste[i]**, i étant l'indice d'ordre dans la liste

Dictionnaire-> liste d'éléments sous la forme d'une paire key:valeur.

`N = {'voiture':'4 roues', 'moto':'2 roues'}`

On accède à la valeur d'un key en l'utilisant comme index (clé de recherche):

`print(N['voiture'])` -> '4 roues'

Fonction-> `def nom_de_ma_fonction(args) :`

on peut ensuite l'appeler dans un programme avec: `nom_de_ma_fonction()`

Module-> `from machine import Pin` -> importe la fonction Pin du module machine

`P0 = Pin(0, Pin.OUT)` -> on paramètre le GPIO 0 en sortie

`P0.value(1)` -> on lui donne la valeur 1 ou état « haut »

G|3 Glossaire informatique 3

Boucles

for degree in range(0,180,1):

XXXXX -> la variable degree va varier de 0 à 180 avec un pas de 1, pour chaque valeur de cette variable, XXXX sera exécuté

global ma_variable -> permet d'avoir accès à ma_variable du programme principal dans la def d'une fonction

def ma_fonction():

global ma_variable

print(ma_variable)

ma_variable += 2

return ma_variable

Return ma_variable -> permet de donner au reste du programme la nouvelle valeur de ma_variable ou d'une nouvelle variable si créée dans la fonction

G|4 Les fonctions du module time

time.time() -> donne le temps présent en ms

time.time_diff(ticks1, ticks2) -> donne la différence entre ticks1 et 2

time.sleep (2) -> met un délai de 2 secondes

utime travaille comme time mais en ms

utime.sleep (100) -> met un délai de 100ms

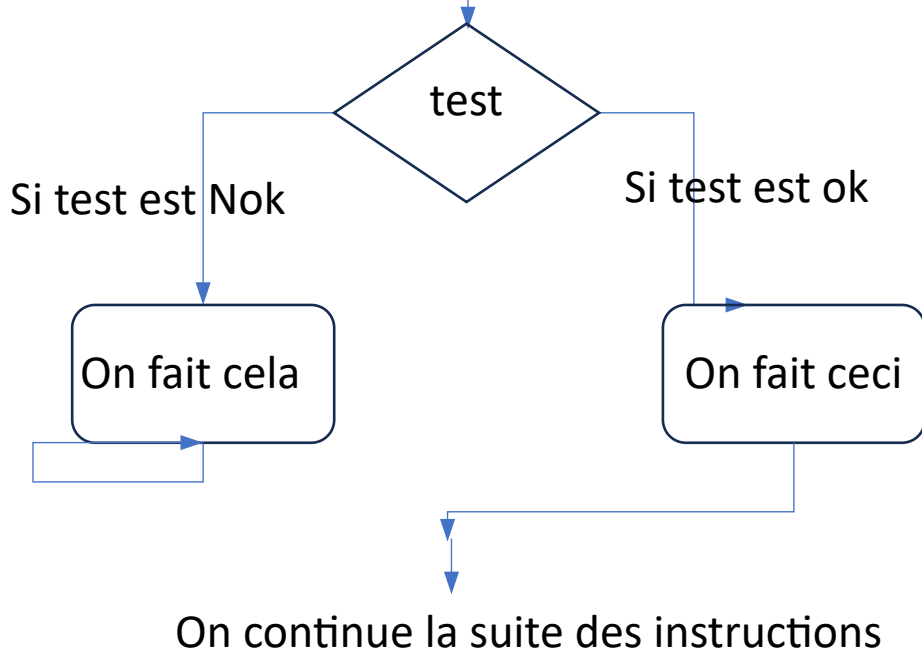
G|5 Glossaire informatique 3

Les tests:

l'instruction **if...else** permet d'exécuter un bloc de code en fonction de la véracité d'une condition, où le corps de l'instruction **if** s'exécute si la condition est **vraie**, et le corps de l'instruction **else** s'exécute si elle est **fausse**, avec une indentation pour définir les blocs de code.

Dans la suite des instructions du programme,

On arrive à un test



Exemple:

Dans le programme A=2

If A > 10:

print('A supérieur à 10')

else:

print ('A inférieur à 10')

On continue d'exécuter les instructions qui suivent

Gl6 Les fonctions du module PWM

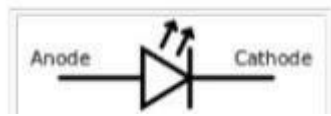
Fonctions utilisées:

```
servoPinPan = PWM(Pin(16))      # servoPinPan est un objet, initialise le Pin GPIO 16 en sortie PWM  
servoPinPan.freq(50)            # Initialise la fréquence des signaux carrés à 50Hz soit 20ms  
servoPinPan.duty_u16(int(newDuty)) # duty_u16 est le rapport cyclique entre le temps du signal carré haut sur sa période, new duty est une variable, le max est à 65535.
```

```
servoPinPan.deinit()           # réinitialise le PWM
```

```
servoX.duty_ns(pulseDX) #commande le rapport cyclique du PWM en nanosecondes suivant la variable pulseDX de l'objet servoX
```

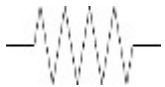

GE1 Glossaire électronique



Symbole de la diode LED



Pile



Résistance



BP

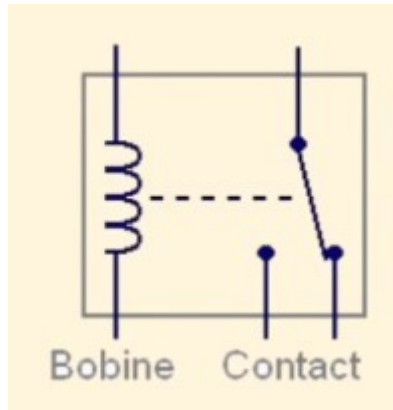
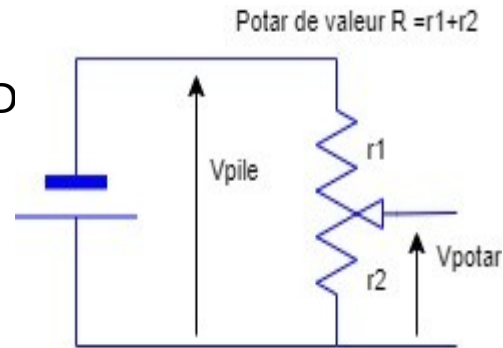


Schéma électrique du relais



$$V_{potar} = V_{pile} * r2 / (r1 + r2)$$



Condensateur
de filtrage signal



Condensateur
Chimique, réserve d'énergie

Couleurs	Tension de seuil ou Vf	If (mA)	Longueur d' onde
Rouge	1,6 V à 2 V	6à20	650 à 660 nm
Jaune	1,8 V à 2 V	6à20	565 à 570 nm
Vert	1,8 V à 2 V	6à20	585 à 590 nm
Bleu	2,7 V à 3,2 V	6à20	470 nm
blanc	3,5 v à 3,8 v	30	

